

Exploring the Effectiveness of Steganography Tools: An Analysis of Data Concealment, Robustness, and Steganalysis (COMP5004)

Submitted by:

KARTHIK ASHOK HONGUNTIKAR

University of Plymouth

Student ID: 10877925

karthik.honguntikar@postgrad.plymouth.ac.uk

Subject: Digital Forensics & Malware Analysis

This is submitted to the University of Plymouth in partial fulfilment of the requirement for the degree of MSc in cybersecurity

University of Plymouth Faculty of School of Engineering, computing, and mathematics

Table of Contents

Introduction	3
Definition of Steganography.....	3
Examples of Steganography tools	3
Open Puff	3
Overview	3
Supported formats.....	4
Features and Capabilities	5
Limitations.....	5
Example of usage.....	6
S-tools	10
Overview.....	10
Features and Capabilities	11
Limitations.....	11
Example of usage.....	12
Steg Hide.....	15
Overview.....	15
Features and Capabilities	16
Limitations.....	17
Example of usage	18

Steganography:

The process of encoding data so that it cannot be easily detected by human investigation into another message or actual item is known as **Steganography**. In computers and electronic contexts, a computer file, message, image, or video is concealed within another file, message, image, or video. The word "steganography" is derived from the Greek word steganós (στεγανός), which means "covered or concealed," and the word "writing," -graphia (γραφία).

These are a few examples of steganography tools:

1. QuickStego
2. Send & Hide
3. OpenStego
4. S-Tools
5. Steghide
6. Stepsolve
7. OpenPuff
8. ExifTool
9. Binwalk

Confidential communication can be concealed using various file formats like DOC, HTML, and pictures, with some programs providing standard steganography and others encrypting data before concealing it.

Open Puff:

Open Puff Steganography and Watermarking, also referred to as Puff or OpenPuff, is a free steganography tool for Microsoft Windows that is being updated as stand-alone software. It was developed by Cosimo Oliboni. The program is notable for being the first steganography tool (December 2004 saw the release of version 1.01).



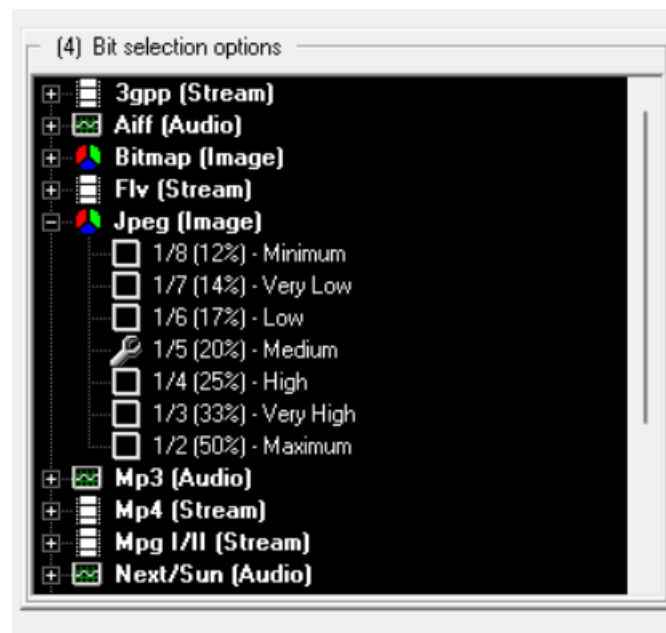
Type	Supported format
Image	BMP, JPG, PCX, PNG & TGA
Audio	AIFF, MP3, SUN & WAV
Flash adobe	PDF, FLV, & SWF
Video	3GP, MP4, MPG, & VOB

The above chart shows the file formats that OpenPuff supports:

OpenPuff enables data concealment in multiple carrier files, creating a carrier chain. The size limit is 256MB, but the ratio of concealed data to carrier file size may vary. Ensuring the carrier's byte capacity is greater than the file size is crucial for effective data concealment.

OpenPuff is a secure steganography software that encrypts and hides data using cryptographic methods, handling various file formats like PDFs, images, audio, and video. Data recovery is effective with correct passwords, but cannot be accessed if passwords are lost or forgotten.

The study evaluated the OpenPuff tool's performance with **JPEG**, **BMP**, and **PNG** image formats, focusing on attributes like steganographic photos. A security steganalysis tool was used to examine pixels and bit values. The PSNR was calculated to defend against image processing attacks, assessing robustness and visual distortion.



OpenPuff uses a novel multi-cryptography approach, utilizing 16 distinct cryptographic algorithms from AES, NESSIE, & CRYPTREC12. Among these algorithms are Anubis, Camellia, Cast-256, Clefia, FROG, Hierocrypt3, Idea-NXT, MARS, RC6, Safer+, SC2000, Serpent, Speed, Twofish, and Unicorn-A23.

Layer	Description
Scrambling	Data that is encrypted is jumbled to destroy any lingering stream patterns.
Whitening	A large quantity of noise is combined with the scrambled data
Encoding:	A non-linear function is used to encode whitened data

Table: The levels of protection and encryption that OpenPuff uses are shown in this table along with their explanations.

Despite being a powerful steganography tool, OpenPuff may have several drawbacks.

- Complexity: It may take some time to become used to the tool's complicated interface and operation.
- Achievement Trade-off: Steganalysis resistance, performance, and security may need to be balanced, with performance potentially impacted by the processing power required for secure data handling.
- User error: Just like any security solution, Open Puff's efficacy may potentially be impacted by improper use. Potential abuse or misconfiguration may result in security flaws.

No tool can guarantee complete security, and its effectiveness depends on its usage. It's advisable to use tools according to best practices and stay updated with the latest versions.

OpenPuff is a professional steganography program suitable for sensitive data transfer, widely favoured by professionals and academics. However, it has limitations like any tool.

Capability:

- Data splitting: OpenPuff divides data among many carriers. Unhiding is only possible with the right carrier sequence.
- Large Data Hiding: If sufficient carriers are available, OpenPuff can conceal data up to 256 MB.
- Pattern Recognition: For OpenPuff MPEG, patterns that can accurately categorize all files (with a success rate of 99.5%) have been discovered.

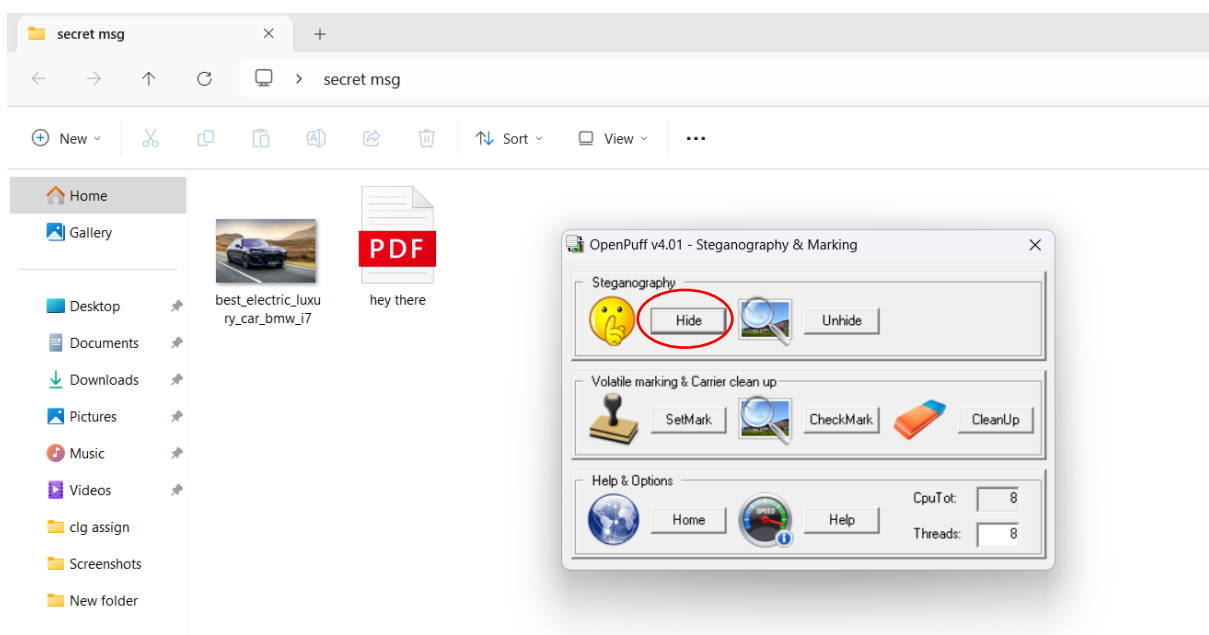
Limitations:

- Carrier File Size: The data that is being concealed must be much less than the carrier files.
- Dependency of Patterns: The patterns identified are contingent upon the specifics of the MPEG file's compression technique.
- Limited Coverage: Patterns matching 100% of the accessible objects were discovered for OpenPuff MP3 files, but they did not match all of them.
- Complexity: It may take some time to become used to the tool's complicated interface and operation.

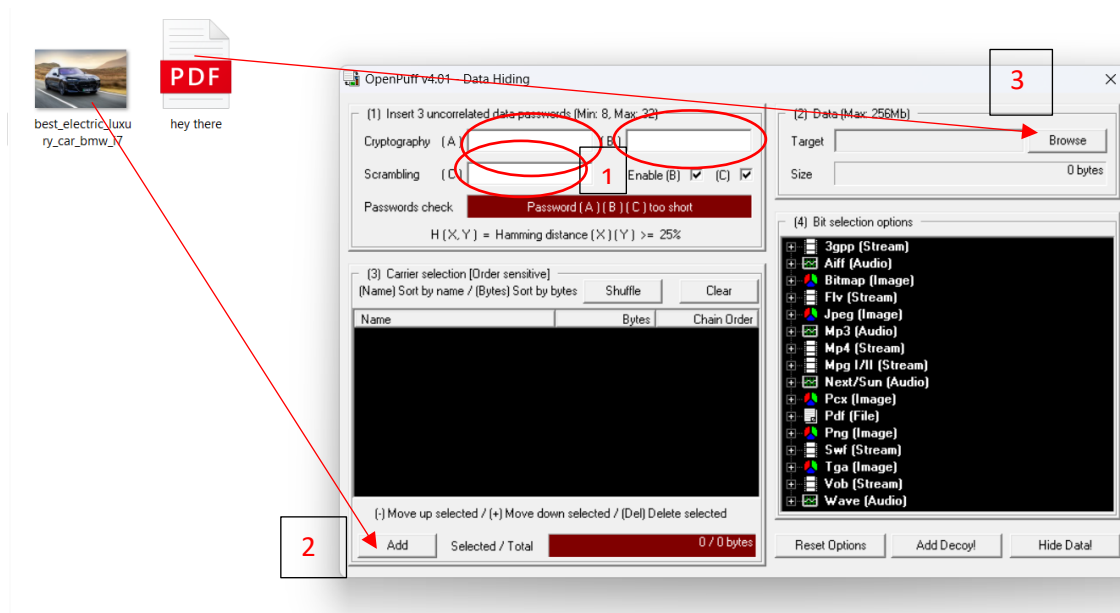
It is crucial to remember that proper usage of the tool can also affect how successful steganalysis is. It is always advised to use these tools according to best practices and to remain current with the most recent versions.

Here is an example where we hide a PDF file inside the JPEG picture file by using the Openpuff Steganography tool

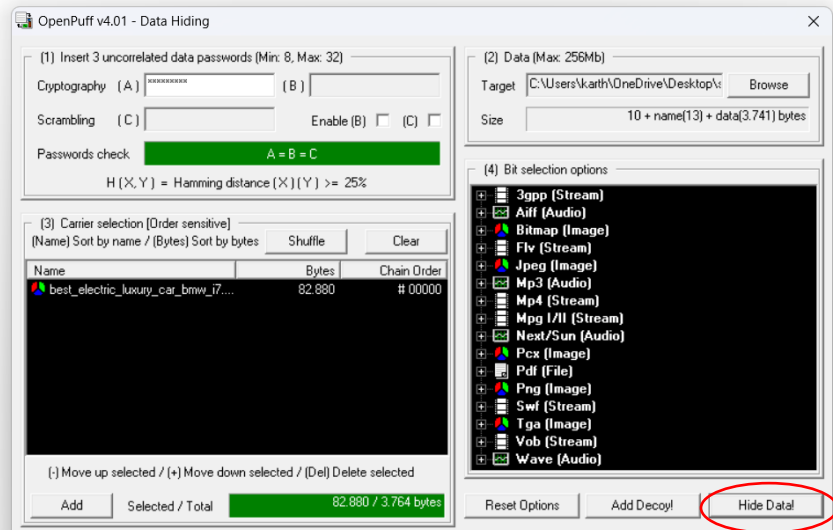
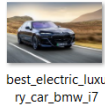
Step 1: Once the tool has been downloaded, open it and launch the executable file. To start the stego procedure, click the "Hide" button from the main page.



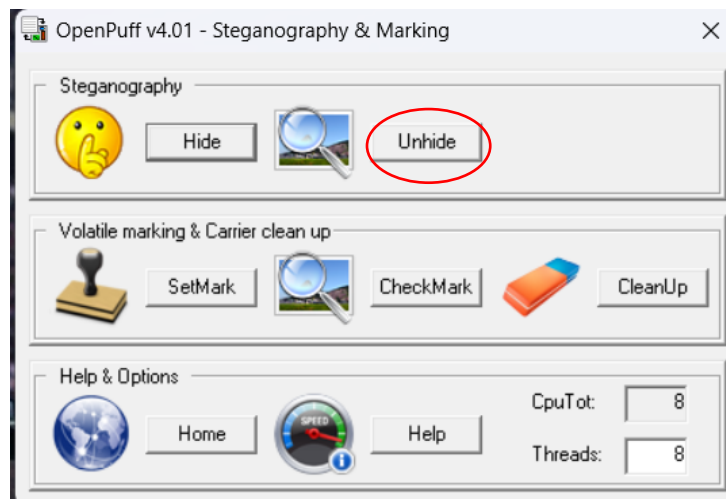
Step 2: Before adding an image file, we must first configure the triple encryption option to conceal hidden messages inside it. The secret message file should then be located and added to the "Target" box. When everything is finished, the steganography procedure will be complete when you click the "Hide Data!" button.



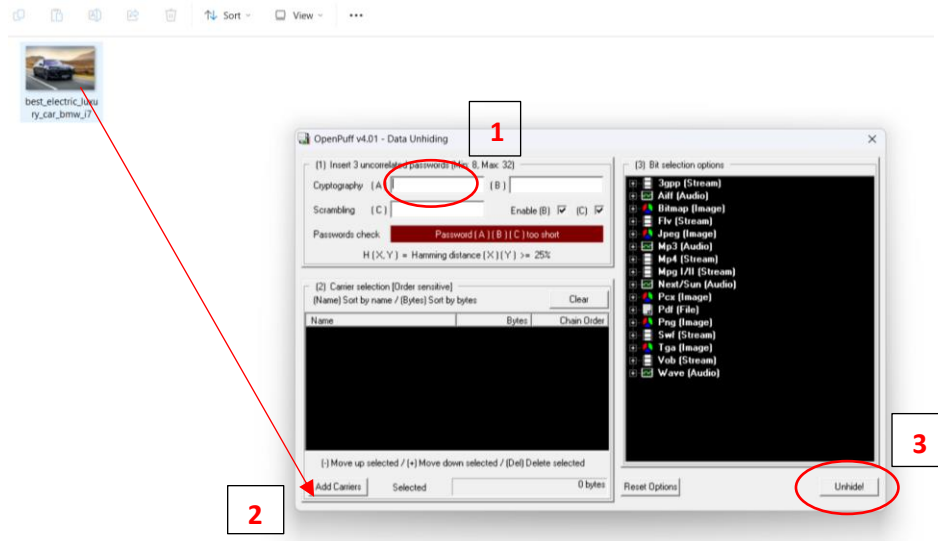
Step 3: Click the Hide data button now. The text file will hide inside the image file.



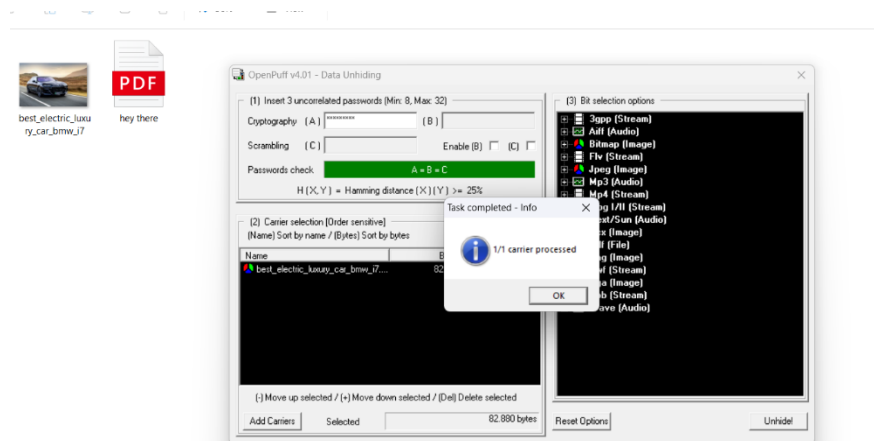
Step 4: Nobody will detect that the picture file has been edited because it still looks like the original. Use OpenPuff to display the message again and select the "Unhide" option.



Step 5: Add the stego picture and select the triple encryption option that you previously used. The secret message file will then be extracted by OpenPuff when you click the "Unhide!" button.



Step 6: We were able to successfully obtain the private data from the Stego picture file, as seen in the screenshot below.



S-Tools:

Andy Brown created S-Tools, a widely used steganography tool that covers sensitive documents in BMP, GIF, and WAV file formats. The tool is user-friendly and uses randomly accessible bits to hide hidden messages. However, the nonlinear addition of a pseudorandom number generator makes the extraction of hidden signals more challenging. This makes it difficult to cover sensitive information in documents.

The carrier file size determines the amount of data that can be concealed using S-Tools, which calculates the available storage bytes after analysing the file. Capacity is determined by Bits-per-pixel (bpp), which is the average number of bits hidden in each pixel.

As concealing capacity increases, hidden data's security, and imperceptibility decrease, making steganography easier to detect. The ideal amount of information to conceal depends on security and invisibility needs, such as a GIF file with 44,984 bytes.

S-Tools is a steganography tool that, based on the file's needs for secrecy and invisibility, may hide data from digital media, mostly audio and image files, utilizing cover files like WAV, GIF, and BMP.

S-Tools offers the option to select between **IDEA**, **DES**, **TripleDES**, and **MDC** encryption algorithms. A passphrase and the chosen encryption scheme need to be kept in mind to successfully extract the secret file.

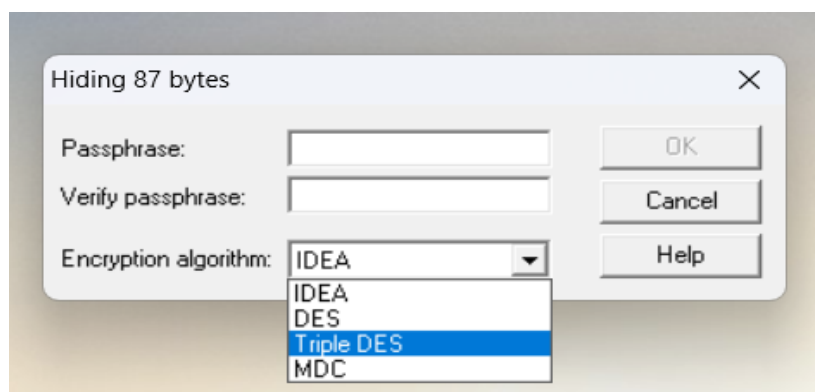


Fig: The above picture shows the available encryption algorithms

S-Tools does, however, have a few possible shortcomings:

- Complexity: The tool requires users to memorize the encryption technique and passphrase, as incorrect algorithm selection during extraction prevents the secret file from being extracted.
- User error: S-Tools' efficacy can be harmed by incorrect usage, just like any other security tool. Security vulnerabilities can result from abuse or incorrect setup.
- Limited File types: S-Tools' usability may be hampered by the fact that it only supports a few numbers of file types, including BMP, GIF, and WAV.

Steganalysis, as used in S-Tools steganography, seeks to locate suspicious packages, ascertain whether a payload is encoded within them, and, if feasible, retrieve that payload.

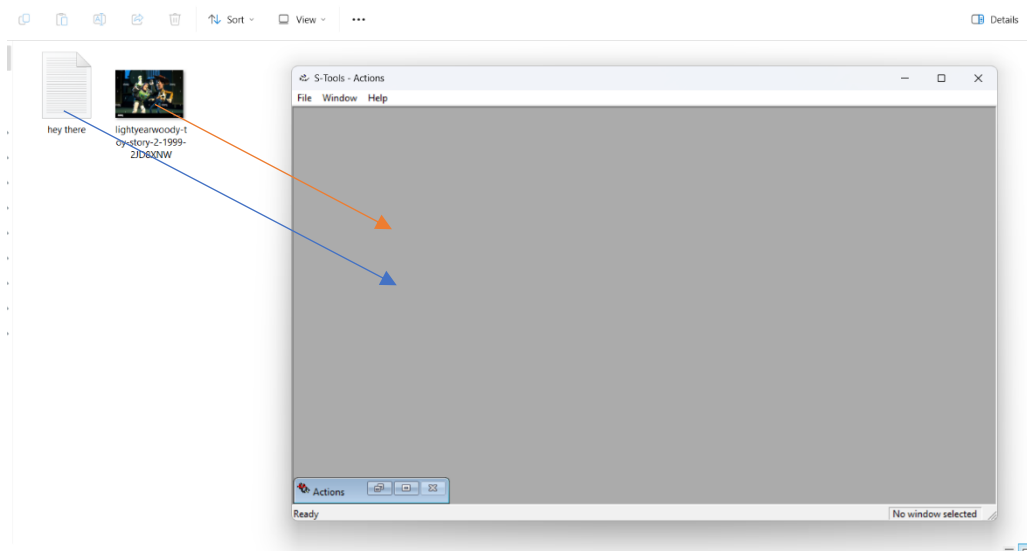
- Detection: The covert communication can be uncovered by using steganalysis techniques to examine suspicious pictures or stego images.
- Recovery: These tools may be able to retrieve the carrier's concealed data as well.
- Adaptability: The Steg analyst and the tools they employ must progress along with steganographic tools like S-Tools.

S-Tools steganography's steganalysis has many limitations:

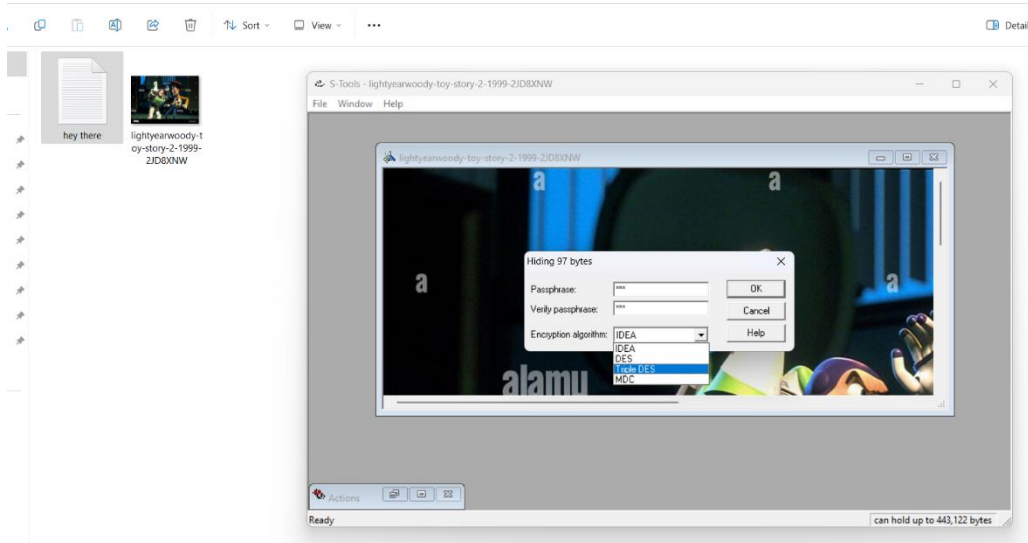
- Evidence: Unless a payload has been fully recovered and encrypted, it is exceedingly risky to get a warrant or take further measures based only on steganalysis evidence.
- Modification Detection: An analyst is only provided with a statistic suggesting that a file has been altered, maybe because of steganographic encoding.
- Progress in Instruments: Steganalysis is challenged by the development of steganographic tools such as S-Tools.

Here is an example where we hide a text file (hey there) inside the JPEG picture file by using the S-tools Steganography tool

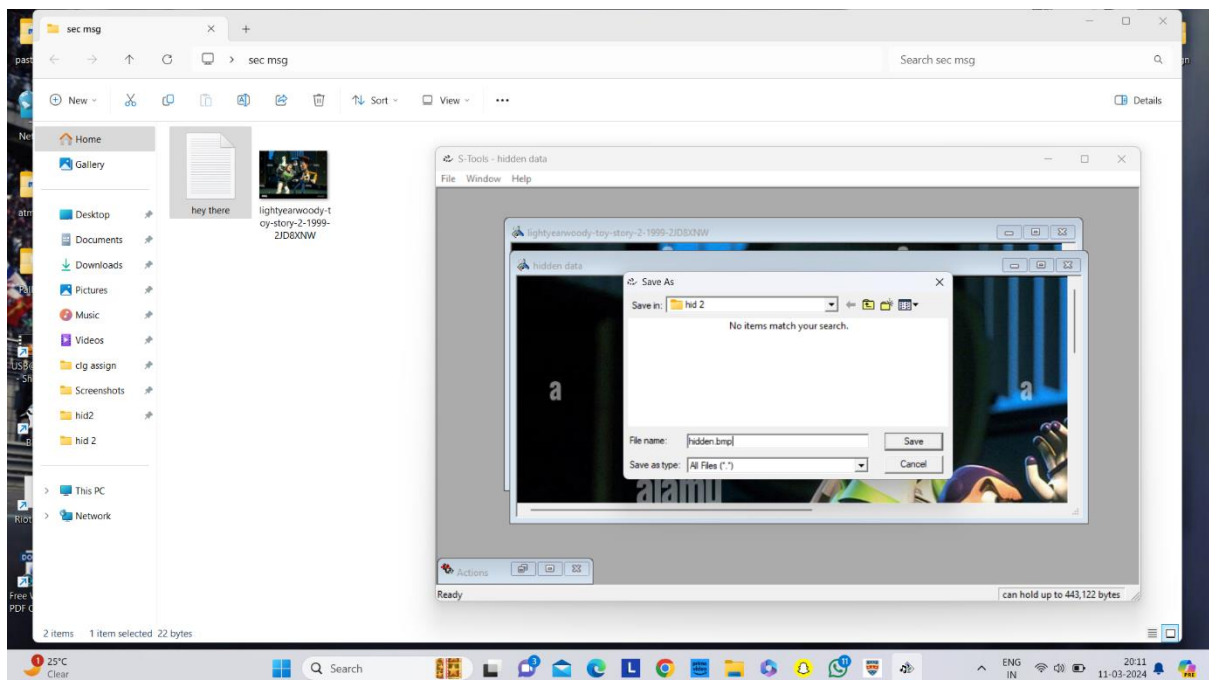
Step 1: It is simple to use this tool—just drag and drop the cover picture file and your secret message file on top of it, as we previously said.



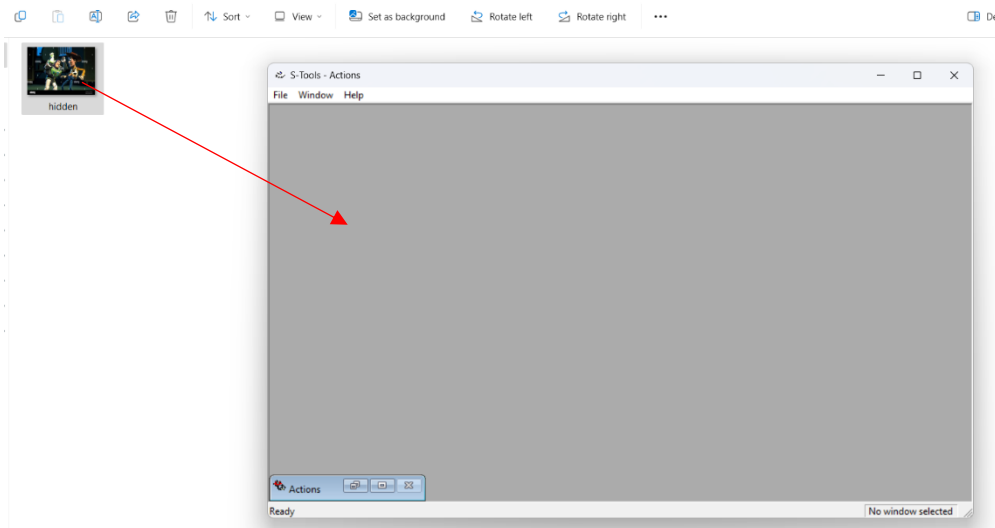
Step 2: You will see a little popup on the main screen requesting you to encrypt the stego file. Click "OK" after creating a new password encryption.



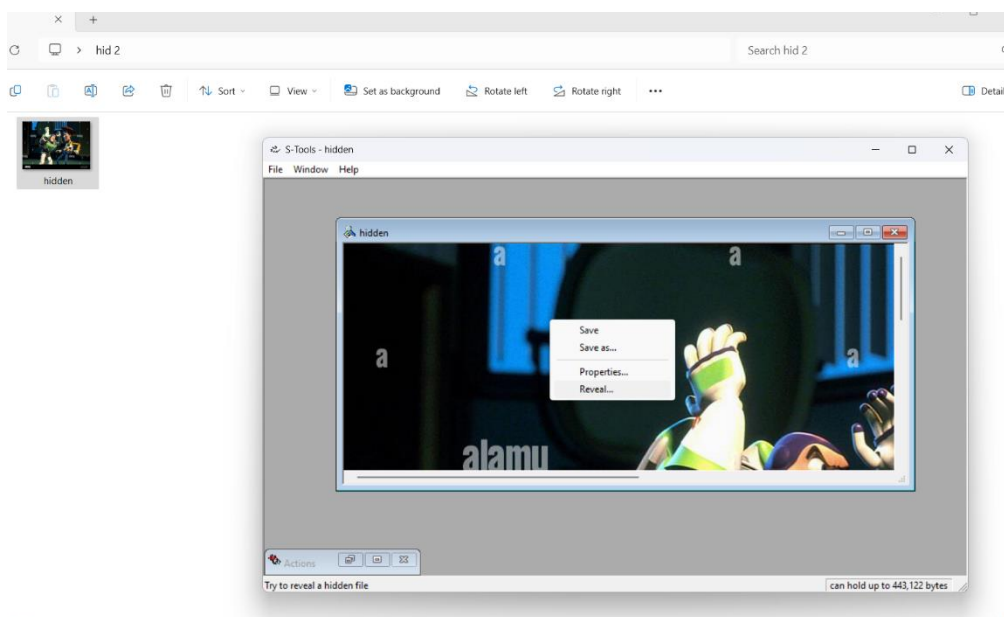
Step 3: Click the "Save" button after selecting the location where the Stego picture should be saved.



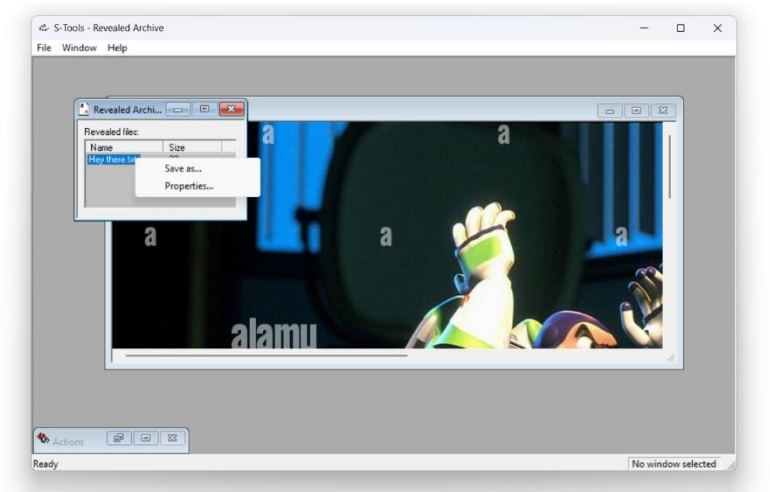
Step 4: Launch S-Tools, drag and drop the file into it, and you will be able to extract the hidden data from the stego picture.



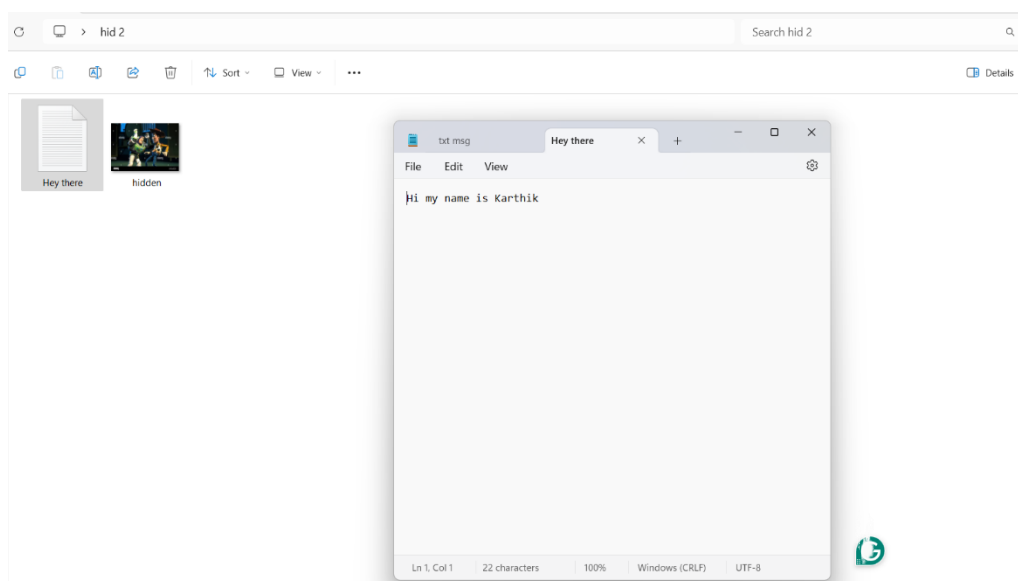
Step 5: Launch S-Tools, drag and drop the file into it, and you will be able to extract the hidden data from the stego picture.
When you do a right-click on the picture, choose "reveal" from the drop-down menu.



Step 6: The secret message file name will be shown in a "Revealed Archive" window that opens. To save the file in a different location, right-click on the file name and choose "Save as..."



Step 7: Once completed, browse to the selected location to view the secret message file. And the secret message can be seen



Steghide:

Steghide is a free steganography application that lets you conceal hidden files within other files. Steghide works by embedding your secret data into an audio, picture, or video file. To everyone else, this carrier file will seem and sound normal. However, it covertly includes your encrypted message or file. The Steghide tool allows you to effortlessly embed your secret information and then extract it with the password. This allows you to securely transfer communications or sensitive data over public networks or any system where privacy is required.

Steghide simplifies steganography techniques on Kali Linux for novices, providing a valuable resource for learning and practising covert communication.

The software is resistant to statistical tests due to its ability to maintain colour and sample frequencies, but its data concealment efficacy may vary based on carrier file type, data size, and steganography parameters.

Scenario	Robustness
File conversion	Varies depending on the conversion procedure and formats used. Converting the carrier file to a different format may destroy the concealed data.
Compression	Steghide is a highly effective compression algorithm, but its recovery may be hindered by severe compression or specific forms.
Noise and signal manipulation	Noise and signal modification can both affect resilience.
Image processing	Certain image processing processes may influence concealed data.

Table: The above chart summarizes Steghide's robustness across various scenarios:

Steghide primarily supports picture and audio file types for steganography. Here are some of the most frequent file formats supported:

Image Format	Audio Format
JPEG, BMP, GIF, PNG, TIFF, PPM	WAV, AU, AIFF, MP3

Steghide is a technique that utilises a multifaceted steganography process.

- Embedding algorithm: Steghide's embedding algorithm uses graph-theoretic methods to embed encrypted and compressed secret data, creating pixel coordinates in cover files, and resisting first-order statistical tests.
- Encryption Algorithm: Steghide uses Rijndael by default, which is AES (the Advanced Encryption Standard) in the cypher block chaining mode, and has a key size of 128 bits.
- Checksum method: To guarantee data integrity, the checksum is computed using the CRC32 method.

Steghide also uses the least significant bit technique, or LSB approach, to conceal data in pictures.

The Steghide steganography tool has a few weaknesses.

- Restricted File Types: There are just a few file types that Steghide can handle. It does not handle video formats; instead, it is mostly used for picture and audio formats.
- Detection: Although Steghide is meant to be hard to find, technological and methodological developments have made it simpler to find concealed data.
- Restricted Data Capacity: Steghide can only conceal a limited amount of data in a file due to its restricted data capacity.

Steganalysis utilizes steganography methods to identify and decode secret data in files, with specific functionalities and constraints within the Steghide steganography instrument framework.

Capability:

- Data Extraction: Tools for steganalysis can extract concealed data from a variety of files, including ones that Steghide has altered.
- Use of Several Approaches: Steganalysis is more successful when it uses a variety of approaches to reveal information that has been hidden.

Limitations:

- Reliant on Steganography Techniques: Steganography techniques significantly influence steganalysis's outcome, with some techniques being more challenging to identify than others.
- Requires Tool Knowledge: Understanding the steganography tools being used, such as Steghide, is necessary for effective steganalysis.
- Possibility of False Positives: Steganalysis methods may produce false positives, identifying hidden data when none is there.

Commands used in Steghide:

Embedding data:

steghide embed -cf <cover_file> -ef<file_to_embed> -sf <output_file> -p<passphrase>

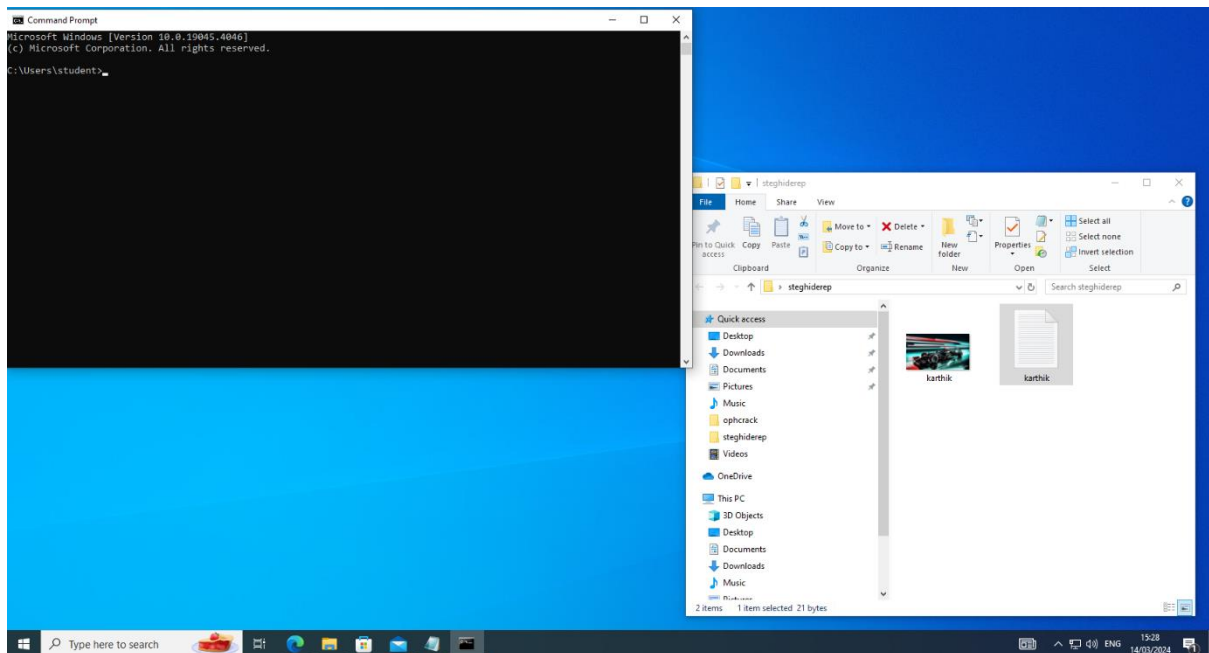
Extracting data:

steghide extract -sf <stego_file> -p<passphrase>

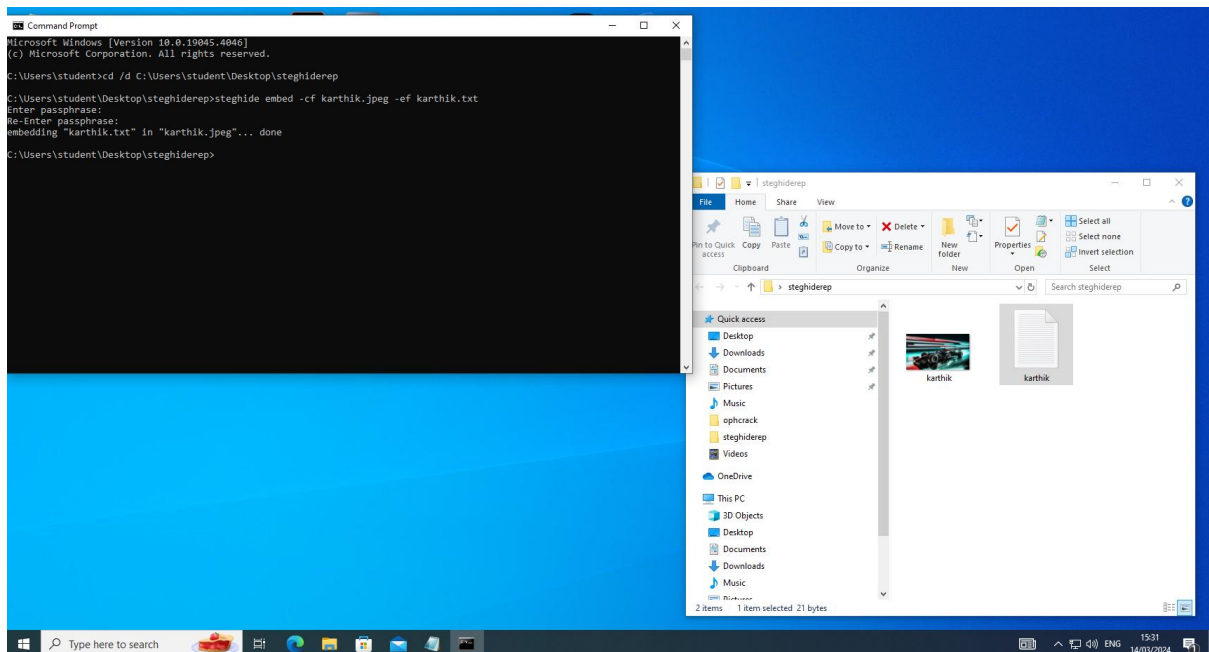
Viewing embedding capacity:

steghide info -cf <cover_file>

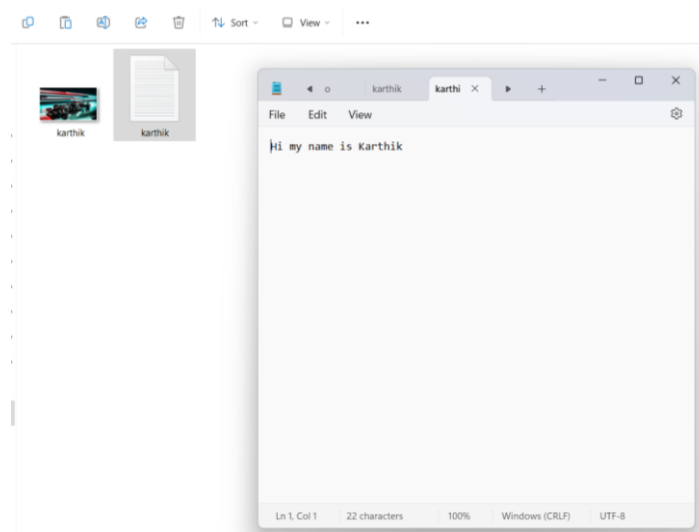
In this example, we will include the "karthik" text file into the "karthik" JPEG picture file. To insert hidden text into a cover file, use the "embed" command. We will use the "-ef" (embed file) operator to indicate the path of the hidden data file and the "-cf" (cover file) operator to find the location of the cover file. We may additionally safeguard our data cryptographically by using a passcode (p), which will be required later during extraction.



Once you press "Enter," the picture file "karthik" picture file in the "/root/Downloads" directory will be modified to include the concealed data.



To extract the concealed data, we will utilize the "extract" command. The executable is loaded with fewer arguments, making it easier to use. First, enter the name of the stego file (-sf) from which to try extraction, followed by a passphrase (p). It is possible to give the name of the output file; the extracted data will be stored by using the "-xf" operator.



```
Command Prompt
Microsoft Windows [Version 10.0.19045.4046]
(c) Microsoft Corporation. All rights reserved.

C:\Users\student>cd /d C:\Users\student\Desktop\steghiderep

C:\Users\student\Desktop\steghiderep>steghide embed -cf karthik.jpeg -ef karthik.txt
Enter passphrase:
Re-Enter passphrase:
embedding "karthik.txt" in "karthik.jpeg"... done

C:\Users\student\Desktop\steghiderep>steghide extract -sf karthik.jpeg
Enter passphrase:
wrote extracted data to "karthik.txt".

C:\Users\student\Desktop\steghiderep>
```

Fig: The above screenshot shows the commands that were used in StegHide

Steganalysis is a powerful tool for locating buried data in unexpected locations, protecting data integrity, and preventing cyberattacks. It uses unique encryption methods for each steganography program. However, successful steganalysis requires proper understanding and application of these tools. As steganography techniques advance, more advanced steganalysis tools must be developed alongside them to ensure effective data protection.